

HelixSkills

HelixSkills

Ein registriertes, verfassungsgestütztes Kompetenzsystem für CLI-AI-Agenten.

Zusammenfassung

HelixSkills ist ein Kompetenzsystem für CLI-AI-Agenten, das das Helix-Constitution als Submodul übernimmt, sodass alle universellen Governance-Regeln bedingungslos gelten. Es bündelt installierbare Agentenkompetenzen, MCP-Toolserver, Claude-Code-Plugins und wiederverwendbare Engines in einem registrierbaren, dokumentierten Katalog.

Kurzbeschreibung

HelixSkills ist ein Kompetenzsystem für CLI-AI-Agenten. Es integriert das Helix-Constitution als Submodul, sodass alle universellen Regeln gelten, und liefert registrierbare Kompetenzen (Aktionspräfix, Medienvalidierung, Multitrack, Sitzungssynchronisation, Lebenszyklus von Arbeitselementen u. v. m.), zwei MCP-Toolserver, zwei Claude-Code-Plugins sowie wiederverwendbare Engines.

Ausführliche Beschreibung

HelixSkills (Repo `skills`, Apache-2.0) ist ein Kompetenzsystem für CLI-AI-Agenten und beginnt mit einer bewussten Umkehrung der üblichen Reihenfolge: Governance steht an erster Stelle, Fähigkeiten folgen danach. Es übernimmt das Helix-Constitution als `constitution/-`-Submodul, sodass jede universelle Regel aus `constitution/CLAUDE.md` und `constitution/Constitution.md` bedingungslos gilt – nicht als Konvention, die ein Agent möglicherweise einhält, sondern als Regelwerk, das physisch in

die Projektstruktur eingebunden ist. Ein Agent, der HelixSkills übernimmt, kann sich nicht von der Verfassung abkoppeln; die Regeln reisen mit dem Code.

Während die meisten „Skill-Frameworks“ mit Abstraktionen handeln, liefert HelixSkills einen konkreten, registrierbaren Katalog, auf den man verweisen und den man installieren kann. Sieben Verfassungskompetenzen werden über `register.sh` installiert: `action-prefix-system`, `media-validator`, `multitrack`, `reporting-workable-items`, `scheduled-work-queue`, `session-sync` und `workable-item-lifecycle` – ein Spektrum von mittlerem bis fortgeschrittenem Niveau, das alles von disziplinierter Aktionsbenennung über Medienvalidierung bis hin zum vollständigen Lebenszyklus einer Arbeitseinheit abdeckt. Zusätzliche Entwurfskompetenzen (Android-Übersicht, Java/Kotlin-Sprache, Linux-Betriebssystem) sind bereits indexiert und bereitgestellt, warten jedoch noch auf ihre Aktivierung. Zwei MCP-Toolserver (`media-validator`, `scheduled-work`) stellen diese Kompetenzen Agenten über Model Context Protocol zur Verfügung, während zwei Claude-Code-Plugins (`helix`, `scheduled-work`) dieselben Fähigkeiten direkt in die Laufzeitumgebung des Agenten einbinden – ein Kompetenzset, das Agenten über die von ihnen genutzte Schnittstelle erreicht.

Unterhalb des Katalogs befinden sich vier wiederverwendbare Engines der ersten Ebene – `continuum` (implementiert) sowie `session_orchestrator`, `token_optimizer` und `clickup_sync` (in Planung) –, die gemeinsame Infrastruktur bereitstellen, damit Kompetenzen nicht ständig dieselben Grundlagen neu erfinden müssen. Allein der `token_optimizer` deklariert eine explizite Abhängigkeitsstruktur, die bis in die Pakete des vasic-digital-Ökosystems (TOON, Embeddings, VectorDB, Normalize, conversation) und HelixDevelopments LLMProvider hineinreicht, sodass seine plattformübergreifenden Verbindungen nachvollziehbar und nicht implizit sind. Das gesamte System wird durch eine disziplinierte Dokumentation abgerundet: ein Kompetenzkatalog, ein automatisch generierter Skill-Graph-Index, detaillierte Seiten pro Repository sowie ein offenes „Lücken & Risiken“-Register, das benennt, was noch nicht umgesetzt ist. Das gesamte System ist über GitHub, GitLab, GitFlic und GitVerse gespiegelt, um Ausfallsicherheit und regionalen Zugriff zu gewährleisten.

Warum wir es entwickelt haben

CLI- und AI-Agenten benötigen Fähigkeiten, die konsistent, regelbasiert und wiederverwendbar sind – keine Ad-hoc-Skripte, die jedes Mal die Regeln neu erfinden. HelixSkills wurde entwickelt, um Agenten ein paketiertes, registrierbares Skill-Set

zur Verfügung zu stellen, das an eine gemeinsame Verfassung gebunden ist. So bleibt das Verhalten über jeden Agenten und jedes Projekt hinweg konsistent und nachvollziehbar.

Warum es ein Game-Changer ist

Es macht die Fähigkeiten von Agenten von Grund auf portabel und regelkonform – nicht durch Disziplin, sondern durch Konstruktion. Jeder Skill ist eine reglementierte, versionierte und installierbare Einheit, die von einem Verfassungs-Submodul gestützt wird. Sobald ein Agent einen Skill registriert, übernimmt er automatisch den kanonischen Regelsatz, ohne Spielraum für Abweichungen. Das ermöglicht etwas, das zuvor nicht praktikabel war: Eine Fähigkeit von einem Agenten oder Projekt auf ein anderes zu übertragen und zu wissen, dass sie bereits an dieselbe Governance gebunden ankommt – bereitgestellt über standardisierte Schnittstellen (MCP-Server und Claude-Code-Plugins) statt eines Haufens maßgeschneiderter Klebeskripte, die jedes Mal die Regeln neu erfinden.

Was innovativ ist

- **Constitution als Submodul:** Universelle Governance-Regeln werden vererbt, nicht kopiert – sie werden in den Baum eingebunden, sodass jeder nutzende Agent an denselben kanonischen Regelsatz gebunden ist. Aktualisierungen fließen aus einer einzigen Quelle, statt aus einem Dutzend veralteter Kopien.
- **Skills als selbstregistrierende Einheiten** (`register.sh`), die in einen automatisch generierten Skill-Graph-Index eingebunden werden. So bleibt der Katalog auffindbar und gerät nie außer Synchronisation mit dem tatsächlich Installierten.
- **Mehrfachschnittstellen-Exposition:** Dasselbe Skill-Set erreicht Agenten über MCP-Tool-Server *und* Claude-Code-Plugins – einmal schreiben, in jeder Laufzeitumgebung einsetzbar, die der Agent nutzt.
- **Wiederverwendbare Tiefen-1-Engines** (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`), die im gesamten Ökosystem geteilt werden. Jede enthält explizite, nachvollziehbare Abhängigkeitsdeklarationen über Repository-Grenzen hinweg, statt versteckter Kopplungen.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Konsistentes und regelkonformes Verhalten von Agenten über viele Skills und Agenten hinweg sicherstellen** – die Governance pro Skill neu zu implementieren, führt mit der Zeit unweigerlich zu Abweichungen. Gelöst durch die Einbindung der Helix-Constitution als Submodul, sodass die Regeln in `constitution/CLAUDE.md` und `constitution/Constitution.md` bedingungslos gelten und aus einer einzigen Upstream-Quelle aktualisiert werden, statt kopiert und dem Verfall überlassen zu werden.
- **Ein wachsendes Skill-Set installierbar und auffindbar machen** – ein Katalog ist nutzlos, wenn niemand findet oder installiert, was darin enthalten ist. Gelöst durch die `register.sh`-Registrierung pro Skill, die jeden Skill bei der Installation einbindet, sowie einen automatisch generierten INDEX-Skill-Graphen und detaillierte Repo-Dokumentationen, sodass die Auffindbarkeit automatisch der Realität folgt.
- **Agenten erreichen, die unterschiedliche Laufzeitumgebungen nutzen** – dieselbe Fähigkeit sollte nicht für jeden Host neu entwickelt werden müssen. Gelöst durch die Bereitstellung eines einzigen Skill-Sets sowohl über MCP-Tool-Server-Definitionen (unter `constitution/mcp/`) als auch über Claude-Code-Plugins (unter `constitution/plugins/`), sodass eine einzige Implementierung über alle Schnittstellen hinweg verfügbar ist.

Technologie-Stack

- **Shell (Primärsprache)** — ausgewählt, weil Installations- und Registrierungstools auf jedem System laufen müssen, auf dem ein Agent existiert, ohne dass zunächst eine Laufzeitumgebung initialisiert werden muss; es treibt `register.sh` und `install_upstreams` an und hält den Einstiegspunkt frei von Abhängigkeiten und portabel.
- **Git-Submodule** — gewählt, um Governance ohne Duplizierung zu übernehmen: Das Helix-Constitution ist unter `constitution/` als lebende Referenz eingebunden, sodass Regelaktualisierungen über einen einzigen Verweis verbreitet werden, statt kopiert und vergessen zu werden.
- **Model Context Protocol (MCP)** — als standardisierte, laufzeitunabhängige Tool-Schnittstelle für Agenten festgelegt; zwei MCP-Server (`media-validator`, `scheduled-work`) sind unter `constitution/mcp/` definiert, um Fähigkeiten als aufrufbare Tools bereitzustellen.
- **Claude-Code-Plugins** — gewählt, um Fähigkeiten ohne zusätzliche Anpassungen direkt in die Agenten-Laufzeitumgebung einzubetten; zwei Plugins (`helix`, `scheduled-`

work) werden unter constitution/plugins/ ausgeliefert und spiegeln die MCP-Oberfläche für einen anderen Host.

- **Wiederverwendbare Engines (continuum, token_optimizer, session_orchestrator, clickup_sync)** — ausgewählt, um gemeinsame Mechanismen aus einzelnen Fähigkeiten herauszulösen und projektübergreifend nutzbar zu machen; token_optimizer ist beispielsweise mit vasic-digital-Paketen (TOON, Embeddings, VectorDB, Normalize, conversation) und HelixDevelopments LLMProvider über deklarierte Abhängigkeiten verknüpft, statt Code zu duplizieren.
- **Mehrhost-Git-Spiegelung (GitHub, GitLab, GitFlic, GitVerse)** — gewählt, damit ein einzelner Host-Ausfall oder eine regionale Sperre den Zugriff nicht unterbricht; dasselbe Repository wird auf vier Plattformen live gehalten, um Ausfallsicherheit und Erreichbarkeit zu gewährleisten.

Status & Transparenzhinweise

- **Status: Beta.** Die sieben Constitution-Fähigkeiten, zwei MCP-Server und zwei Plugins sind ausgeliefert; die Entwurfsfähigkeiten sind indexiert und warten auf Aktivierung, und drei der vier Engines der ersten Tiefe (session_orchestrator, token_optimizer, clickup_sync) befinden sich noch in der Konzeptionsphase.
- Die README bezeichnet das Projekt als helix_skills; der kanonische GitHub-Pfad lautet HelixDevelopment/skills. Die in der README aufgeführte Anzahl der gefundenen Ergebnisse ist eine selbst gemeldete Zahl.

Prioritätsstufe: Helix-primär.